TrustPay Guardian: Fraud-Aware Digital Banking Platform — Antigravity Step-by-Step Prompt Guide

Companion to the PRD (`docs/PRD.md`). Project name (use exactly): **TrustPay Guardian: Fraud-Aware Digital Banking Platform**. Every phase below has a ready-to-paste prompt, a list of checks that must pass, and a commit message. Do the phases in order and never skip the checks.

How to use this guide

1. Create an empty GitHub repository named `trustpay-guardian`, clone it, and open the folder in Antigravity.
2. Export the PRD document as Markdown and save it as `docs/PRD.md` in the repo. Commit it.
3. At the start of **every new session or conversation**, paste the **Master prompt** below, then paste the prompt for the current phase.
4. Ask for a plan first and approve it before the agent writes code (use planning mode if your Antigravity version has it).
5. When the agent finishes, run the checks yourself. Do not trust 'done' without seeing the commands and results.
6. When every check passes, commit on a feature branch, push, open a pull request and merge into `develop`. Merge `develop` into `main` at the tags (`v0.1.0` after Phase 8, `v1.0.0` after Phase 15).
7. If a phase fails or the agent drifts, use the **Recovery prompts** at the end. Do not start the next phase until the current one is green.
8. You must understand what you submit. After each phase, use the 'Explain this to me' prompt on any file you cannot explain.

Branch naming: `feature/phase-03-auth` , `feature/phase-06-fraud-engine` , and so on. **Commit style:** Conventional Commits.

Version targets. Spring Boot (4.1 line) and MySQL (9.7 LTS) were checked against current release pages on 17 October 2026. Use Java 25 LTS, Node.js 24 LTS or newer Active LTS, React 19.x, React Router 7, and the latest stable Vite, TypeScript, Bootstrap 5.3.x, Maven and library versions, confirmed at install time. If the newest version of a library breaks with Spring Boot 4.1 or Java 25, pin the newest compatible one and note it in `docs/DECISIONS.md` .

Master prompt (paste at the start of every session)

```
You are a senior Java Full Stack engineer building the project 'TrustPay
Guardian: Fraud-Aware Digital Banking Platform'. The full specification is in
docs/PRD.md. Read it completely before doing anything.

Rules:
- Follow the PRD exactly: names, packages, folder structure, API paths,
tables, security rules and UI requirements. If something is unclear, choose
the simplest option that matches the Java Full Stack syllabus and add one
line to docs/DECISIONS.md.
- Work only on the phase I give you. Do not start later phases.
- Before writing code, give me a short plan (files to create or change,
commands to run) and wait for my approval.
- Never guess dependency names or versions. Check official documentation,
Maven Central or npm. Use Java 25 (LTS), Spring Boot 4.1.x, Maven, MySQL 9.7
LTS, and the latest stable React 19.x with TypeScript and Vite. For every
other tool and library use the latest stable release available today,
verified before you install it.
- Code must be simple enough for a student to explain: package-by-feature,
small methods, no Lombok, records for DTOs, short comments that explain why.
- Never hard-code secrets. Use environment variables and .env.example.
- When finished, run the build and tests, then report: (1) files created or
changed, (2) commands you ran and their real output summary, (3) anything
that differs from the PRD, (4) how I can verify it manually.
- Do not claim anything works unless you ran it and saw it work.
```

Phase 0 — Repository scaffolding and tooling

Goal: a clean repo skeleton that builds and starts, with MySQL running in Docker.

```
PHASE 0. Create the repository skeleton exactly as in PRD section 11 (only the folders and files needed for now):
1. Root: README.md (skeleton with the project title), LICENSE (MIT), .gitignore (Java/Maven, Node, IDE, OS, .env, logs, target, node_modules, dist), .gitattributes, .env.example (all variables from PRD section 12 with safe placeholders), CONTRIBUTING.md, docs/DECISIONS.md, .github/pull_request_template.md and .github/ISSUE_TEMPLATE/ (bug and feature).
2. backend/: a Maven project with the wrapper, group com.trustpay, artifact guardian, base package com.trustpay.guardian, Java 25 (my machine runs JDK 25, so set java.version and maven.compiler.release to 25 and use JDK 25 in Docker and CI), Spring Boot 4.1.x. Dependencies: Web MVC, Validation, Data JPA, Security, MySQL driver, Flyway (with the MySQL module), Actuator, Mail, springdoc-openapi (version compatible with Boot 4), and the test starters. Add application.yml, application-dev.yml and application-prod.yml reading values from environment variables.
3. frontend/: Vite React + TypeScript project (strict mode). Install react-router@7, axios, bootstrap@5.3, bootstrap-icons, react-hook-form, zod, @hookform/resolvers, @reduxjs/toolkit, react-redux, recharts, plus vitest and testing-library packages. Show a simple placeholder page.
4. docker-compose.yml with only mysql (9.7 LTS, named volume, healthcheck) and mailhog for now.
Do not write business code yet.
```

Checks

- `./mvnw clean verify` passes in `backend/`.
- `docker compose up -d mysql mailhog` shows both containers healthy.
- java -version and ./mvnw -v report Java 25, and the backend starts with the dev profile and `GET /actuator/health` returns `UP` (database included).
- `npm run build` and `npm run dev` work in `frontend/`.
- `git status` shows no `.env`, `target/` or `node_modules/`.

Commit: `chore: scaffold repository, backend and frontend skeleton`

Phase 1 — Database schema and JPA entities

```
PHASE 1. Implement PRD section 6 exactly.
- Flyway migrations V1 to V6 in backend/src/main/resources/db/migration with the exact table names, columns, types, CHECK constraints, unique constraints, foreign keys and indexes from the PRD. V6 seeds the 7 fraud rules (R1 to R7 with default points and parameters from PRD section 5) and the 6 system settings.
- JPA entities, enums (@Enumerated STRING) and Spring Data repositories in their feature packages (user, account, beneficiary, transaction, fraud/config, fraud/alert, auth, notification, audit, admin). Use BigDecimal for money, a @Version field on Account, LAZY relationships, and explicit equals/hashCode on id only.
- Set spring.jpa.hibernate.ddl-auto=validate.
- Add one Testcontainers (MySQL) test class that starts a real MySQL, runs the migrations, and saves then reads one row of every entity. Add a test proving the CHECK constraint rejects a negative account balance.
- Generate docs/diagrams/erd.md (a Mermaid ER diagram matching the tables).
Do not create controllers or services yet.
```

Checks

- Flyway applies V1 to V6 on an empty database; the application starts with `ddl-auto=validate` (no schema mismatch).
- `fraud_rules` has 7 rows and `system_settings` has 6 rows.
- All repository tests pass, including the negative balance test.
- The ERD matches the PRD tables.

Commit: `feat(db): add flyway schema, entities and repositories`

Phase 2 — Common foundation

```
PHASE 2. Build the shared foundation described in PRD sections 7, 8 and 10.
- common/exception: NotFoundException, ConflictException,
BusinessRuleException and GlobalExceptionHandler (@RestControllerAdvice)
returning RFC 7807 ProblemDetail plus fieldErrors and requestId. Map
validation errors to 400, business rules to 422, conflicts to 409, not found
to 404, unexpected errors to 500 without stack traces.
- common/web: RequestIdFilter (puts a request id in MDC and the X-Request-Id
response header) and PageResponse<T>. common/util: MaskingUtils (account
number, email) and MoneyUtils.
- logback-spring.xml (readable console pattern with requestId in dev) and a
rule that passwords, tokens and secrets are never logged.
- config: AppProperties (@ConfigurationProperties for jwt, cors, cookie,
limits), CorsConfig (only FRONTEND_ORIGIN, credentials allowed),
OpenApiConfig (title 'TrustPay Guardian API', bearer scheme),
SchedulingConfig.
- audit: AuditService with a record(actor, action, entityType, entityId,
details) method that writes append-only audit_logs rows (read the IP from the
request when available).
- Tests: MaskingUtils unit tests; a MockMvc test showing a validation error
returns ProblemDetail with fieldErrors and requestId.
```

Checks

- Swagger UI opens at `/swagger-ui.html`.
- A deliberately invalid request returns a ProblemDetail with `fieldErrors` and a `requestId` that also appears in the log line.
- No stack trace is returned to the client.
- Tests pass.

Commit: `feat(common): add error handling, request id, logging, config and audit service`

Phase 3 — Authentication and role-based access control

```
PHASE 3. Implement PRD sections 4.1 and 8 completely.
- Endpoints: POST /auth/register, /auth/login, /auth/refresh, /auth/logout,
GET /auth/me, and [S] POST /auth/change-password.
- security: SecurityConfig (stateless, URL rules by role,
@EnableMethodSecurity), JwtService (HS256, 15 min, claims
sub/role/iat/exp/jti; refuse to start if JWT_SECRET is missing or shorter
than 32 bytes), JwtAuthenticationFilter (OncePerRequestFilter),
AppUserDetailsService, RestAuthenticationEntryPoint (401 ProblemDetail),
RestAccessDeniedHandler (403 ProblemDetail), RateLimitFilter with Bucket4j
(login 10 per minute per IP, transfer 20 per minute per user, 429 with Retry-
After), CurrentUser helper.
- Passwords: BCrypt strength 12. Lockout after 5 failures for 15 minutes with
one generic error message.
- Refresh tokens: random 256-bit value, stored only as a SHA-256 hash in
refresh_tokens, 7 days, rotated on each use, reuse of a revoked token revokes
all tokens of that user, sent as an HttpOnly SameSite=Strict cookie (Secure
controlled by COOKIE_SECURE).
- Registration creates a CUSTOMER only (never accept a role from the client)
and a default SAVINGS account funded with DEMO_OPENING_BALANCE (add a minimal
AccountService.createDefaultAccount for this).
- DevDataSeeder (dev profile only): 1 admin, 1 analyst and 3 customers from
PRD section 16, with passwords taken from DEV_SEED_PASSWORD.
- Audit events for register, login success and failure, logout, password
change.
- Tests: JwtServiceTest, AuthServiceTest (lockout, rotation, reuse
detection), MockMvc tests (400, 401, 403), and an integration test: register,
login, me, refresh, logout.
```

Checks (try them in Swagger or curl)

- Register works; a duplicate email returns 409; a weak password returns 400 with field errors.
- Login returns an access token and sets the refresh cookie (HttpOnly).
- Five wrong passwords lock the account; the message is identical for every failure.
- `/auth/me` without a token returns 401. A customer token on `/admin/users` returns 403.
- Refresh rotates the token; using the old refresh token again fails and revokes the session.
- The application refuses to start without `JWT_SECRET`.
- The database stores a BCrypt hash, and logs contain no passwords or tokens.

Commit: `feat(auth): add JWT authentication, refresh rotation, lockout and RBAC`

Phase 4 — Accounts and beneficiaries

```
PHASE 4. Implement PRD sections 4.2 and 4.3.
- Accounts: AccountService, AccountController, AccountNumberGenerator (TPG + 10 digits using SecureRandom, retry on collision), list own accounts (masked), open an extra account (max 3), account detail, statement with date range and pagination, available balance (balance minus outgoing PENDING_REVIEW transfers). A FROZEN account cannot send or receive.
- Beneficiaries: lookup by account number (return only a masked holder name), add (account must exist, not own account, unique per user), list, delete.
- Ownership checks live in the service layer for every lookup. Another user's resource returns 404 (not 403).
- DTO records for every request and response, Bean Validation, OpenAPI annotations with examples.
- Tests: unit tests for the services, MockMvc tests, and an explicit IDOR test where user A tries to read user B's account, statement and beneficiary.
```

Checks

- A new customer sees exactly one account; opening a 4th account returns 422.
- Account numbers in lists look like `TPG******1234`.
- User A requesting user B's account id gets 404.
- Adding yourself or a duplicate as a beneficiary is rejected.
- All tests pass.

Commit: `feat(accounts): add accounts, statements and beneficiaries`

Phase 5 — Transfers, deposits and history (money movement)

```
PHASE 5. Implement PRD section 4.4. Fraud scoring is added in Phase 6, so
define an interface RiskAssessor { RiskDecision assess(TransactionContext
ctx); } and wire a temporary implementation that always returns score 0 (mark
it clearly with a TODO for Phase 6).
- POST /transactions/transfer: requires the Idempotency-Key header (UUID).
The same key from the same user returns the original result and never debits
twice. Validate amount (min 1, max MAX_TRANSFER_AMOUNT, 2 decimals),
beneficiary ownership, account status, available balance and the account's
daily limit (422 with clear messages).
- Atomic @Transactional execution. Lock both account rows with pessimistic
write locks in ascending id order. Generate the reference TXN- plus 12 random
uppercase alphanumerics.
- POST /transactions/deposit (demo add money, max MAX_DEPOSIT_AMOUNT).
- GET /transactions with filters (type, status, date range, min and max
amount, reference search) and pagination, and GET /transactions/{reference}
(owner only).
- Create in-app notifications for sender and receiver on completion. Write
audit events.
- Tests: success, insufficient funds, frozen account, daily limit, idempotent
replay, and a concurrency test running 20 parallel transfers that proves no
account is overdrawn and the total money in the system is unchanged.
```

Checks

- A transfer moves exactly the amount; both balances and both statements are correct.
- Sending the same request twice with one Idempotency-Key debits once and returns the same reference.
- The concurrency test passes repeatedly (run it 5 times).
- A transfer to a frozen account fails with 422 and no balance changes.
- Money columns are DECIMAL and Java uses BigDecimal everywhere.

Commit: `feat(transactions): add atomic idempotent transfers, deposits and history`

Phase 6 — Fraud detection engine and analyst workflow

```
PHASE 6. Implement PRD section 5 completely and replace the temporary
RiskAssessor.
- fraud/engine: FraudRule interface (code(), evaluate(TransactionContext)
returning Optional<RuleHit>), TransactionContext, RuleHit, RiskDecision,
FraudEngine (receives List<FraudRule> by constructor injection, sums points
capped at 100, applies the bands from REVIEW_THRESHOLD and BLOCK_THRESHOLD
settings).
- fraud/rules: the seven rules R1 to R7 as separate @Component classes with
the exact behaviour in the PRD table. Parameters, points and enabled flags
come from fraud_rules via FraudRuleConfigService (cache them and clear the
cache when an admin updates a rule). Use indexed queries only.
- Integrate the engine into TransferService: score before moving money.
COMPLETED moves money; PENDING_REVIEW and BLOCKED do not. Persist the
fraud_alert and fraud_alert_hits (with readable details) for flagged
transfers. Notify the customer with a generic message and write audit events.
- fraud/alert: FraudAlertService and FraudAlertController. GET /fraud/alerts
and /fraud/alerts/{id} for ANALYST and ADMIN (read-only for admin), POST
release and confirm-block for ANALYST only with required notes. Release re-
checks balance and status, then executes the transfer in one transaction; if
funds are missing the transaction becomes FAILED. A second resolution attempt
returns 409. GET /fraud/stats.
- Tests: every rule at, just below and just above its boundary; engine
scoring and bands; a full integration test of the demo script in PRD section
16 (score 0 completes, 60 is held, 70 or more is blocked, release moves
money, confirm-block does not); role tests (customer 403, admin cannot
resolve).
```

Checks

- A ₹500 transfer completes with score 0.
- A ₹60,000 transfer to a beneficiary added less than 24 hours ago scores 60 and is `PENDING_REVIEW`; no balance changes.
- Rapid repeated large transfers reach 70 or more and are `BLOCKED`.
- Analyst release completes the transfer and notifies the customer; confirm-block keeps it blocked.
- The customer never sees rule names or scores; the analyst sees them.
- Disabling a rule in the database changes the next score (cache cleared by the service).
- Scoring runs under 200 ms (log the duration once to confirm).

Commit: `feat(fraud): add strategy-based fraud engine, alerts and analyst workflow`

Phase 7 — Notifications, admin APIs and statistics

```
PHASE 7. Implement PRD sections 4.5, 4.6 and 4.7.
- Notifications API (list, mark read, mark all read) and [S] email for held
or blocked transfers through MailHog.
- Admin APIs: users (list with search and filters, create analyst or admin,
change role, enable or disable; an admin cannot demote or disable
themselves), accounts (list, freeze, unfreeze with notifications),
transactions (all, with filters), fraud rules (list, update with validation 0
to 100 and cache clear), settings (list, update with validation), audit logs
(filters and pagination, read-only), dashboard statistics (users, accounts,
volume, alerts by status and by rule, transactions per day for 14 days).
- GET /public/stats with aggregate, non-sensitive counters only.
- Every admin action writes an audit event.
- Tests: a parameterised MockMvc test that calls every endpoint with each of
the four identities (anonymous, customer, analyst, admin) and asserts the
expected 200, 401 or 403 from the PRD permission table.
```

Checks

- The role matrix test passes for every endpoint.
- An admin cannot disable themselves; changing a rule changes `updated_by` and writes an audit row.
- `/public/stats` reveals no personal data.
- Frozen accounts are blocked from transfers immediately.

Commit: `feat(admin): add notifications, admin APIs, audit viewer and statistics`

Phase 8 — Backend hardening, quality and release v0.1.0

```
PHASE 8. Harden and measure the backend.
1. Add JaCoCo and report coverage. Raise coverage on the service, fraud and
security packages to at least 80% with meaningful tests (not empty ones).
2. Add sonar-project.properties and make the project ready for a SonarQube
scan; fix blocker and critical issues and the top code smells.
3. Check for N+1 queries on every list endpoint (enable Hibernate statistics
or SQL logging in the test profile) and fix them with join fetch or DTO
projections.
4. Do a written security self-review against the OWASP Top 10 and PRD section
8. Test explicitly for IDOR, mass assignment (role in the register body),
missing rate limits, sensitive data in logs and overly permissive CORS. Fix
what you find and list the findings in docs/SECURITY_REVIEW.md.
5. Make sure every endpoint in Swagger has descriptions, request and response
examples, and the correct security requirement.
6. Add a Backend section to the README: prerequisites, environment variables,
how to run, how to test, demo accounts for dev, and the demo script.
```

Checks

- `./mvnw clean verify` passes; the JaCoCo report meets the targets.
- No N+1 query pattern remains in list endpoints.
- `docs/SECURITY_REVIEW.md` lists what was tested and the result.
- Swagger documents every endpoint.

Commit and tag: `test: raise coverage, fix quality and security findings` then tag `v0.1.0`.

Phase 9 — Frontend foundation: theme, layout, API client and authentication

Goal: the app shell, dark and light themes, secure session handling, and working Login and Register pages.

```
PHASE 9. Build the frontend foundation described in PRD section 9. Backend is already running on port 8080.
- Vite config: dev proxy for /api to http://localhost:8080, strict TypeScript, ESLint and Prettier.
- styles: variables.css (design tokens for colours, spacing, radius, shadows), theme.css, custom.css. Use Bootstrap 5.3 with data-bs-theme plus CSS variables. ThemeContext with a light and dark toggle, default from prefers-color-scheme, saved as tpg-theme, and a tiny inline script in index.html to avoid a flash of the wrong theme.
- api/axiosClient.ts: base URL from VITE_API_BASE_URL, withCredentials, request interceptor adding the access token (kept in memory only, never localStorage), response interceptor that on 401 tries one /auth/refresh and retries, otherwise logs out; 403 goes to /403; 429 shows a toast; 5xx shows a generic toast.
- context/AuthContext.tsx: user, role, login, register, logout, and a silent refresh on page load.
- routes: AppRoutes with lazy-loaded route groups, ProtectedRoute, RoleRoute, and role-based redirects after login (customer to /app/dashboard, analyst to /analyst/dashboard, admin to /admin/dashboard).
- components/layout: PublicNavbar, AppShell (responsive sidebar and top bar per role with theme toggle and user menu), Footer. components/common: Toast system, ConfirmModal, Skeleton, EmptyState, ErrorBoundary, PageHeader.
- types/ for every API model. pages/errors: 403 and 404.
- pages/auth: Login and Register using React Hook Form and Zod, show or hide password, password-strength meter, server field errors mapped onto the form.
- Use a temporary placeholder dashboard page per role.
```

Checks

- Register then login works against the real backend; a wrong password shows a friendly error; five failures show the lockout message.
- Refreshing the browser keeps you logged in (silent refresh); logout clears it.
- The access token is not in localStorage or sessionStorage (check DevTools); the refresh cookie is HttpOnly.
- Typing a protected URL while logged out redirects to login; a customer opening `/admin/dashboard` sees the 403 page.
- The theme toggle works, persists after reload and shows no flash.
- `npm run lint`, `tsc --noEmit` and `npm run build` pass.

Commit: `feat(frontend): add theme system, layout, API client and auth pages`

Phase 10 — Public home page with videos and animations

```
PHASE 10. Build the public Home page exactly as described in PRD section 9
(Home page).
- Sections in order: sticky navbar, hero with a muted looping background
video and poster, animated rotating headline, call-to-action buttons;
animated counters from GET /public/stats; 'How fraud detection works'
animated pipeline (CSS or SVG); feature cards; security highlights; video
gallery with a modal player; testimonial carousel labelled as demo content;
FAQ accordion; final call to action; footer.
- Create hooks useInView (IntersectionObserver) and useCountUp, and use them
for scroll-reveal and counters. Respect prefers-reduced-motion (poster image
instead of video, no animations).
- Videos are loaded from frontend/public/media/ (hero.mp4, hero-poster.jpg,
how-it-works.mp4, security.mp4, fraud-demo.mp4 and posters). Do not generate
or invent video files. Create frontend/public/media/README.md that lists
exactly which video to record or download, the recommended length (10 to 30
seconds), resolution (1280x720), maximum size (8 MB), H.264 encoding, and
reminds me to note the licence of any stock clip.
- If a video file is missing or fails to load, show a gradient and the poster
with no console errors. If /public/stats fails, show sensible fallback
numbers silently.
- Fully responsive from 360 px, accessible (alt text, aria labels, keyboard-
accessible modal with focus trap and Escape to close), and correct in both
themes.
```

Checks

- With and without the video files, the page looks good and the console shows no errors.
- Counters animate once when scrolled into view; reduced-motion shows no animation.
- The video modal opens, plays, closes with Escape and returns focus.
- Both themes look right at 360 px, 768 px and 1280 px.
- Lighthouse accessibility score is at least 90 for the home page.

Before moving on (you): record or download your videos, compress them, put them in `frontend/public/media/` with the exact file names, and note the licences in the README.

Commit: `feat(home): add animated home page with video hero, gallery and counters`

Phase 11 — Customer area

```
PHASE 11. Build the customer pages from PRD section 9 under /app. Use the real API for everything.
- Dashboard: greeting, total balance, account cards, quick actions (Transfer, Add money, Add beneficiary), income versus spending chart for 6 months (Recharts, themed), last 5 transactions, notification bell with unread count polling every 30 seconds (Redux Toolkit notifications slice), security card (last login, Fraud protection: Active).
- Accounts and statement: filters, pagination, page size, [S] CSV export.
- Transfer: select account and beneficiary, amount, note, a review and confirm step, then a result modal for Completed, Held for review (with reference and a clear explanation) or Blocked. Generate one UUID Idempotency-Key per form submission and reuse it on retry. Disable the submit button while sending.
- Beneficiaries: lookup by account number, confirm the masked holder name, add, delete with confirmation.
- Transactions history with filters and pagination, transaction detail drawer, notifications page, profile and settings (change password, theme).
- Every page: skeleton loading, empty state, error state with retry, toasts, accessible forms, both themes.
```

Checks (use the seeded customers)

- Complete the demo script from the customer side: ₹500 completes; ₹60,000 to a new beneficiary shows ‘Held for review’.
- Double-clicking Send creates one transaction.
- Another customer’s data never appears; manipulating ids in the URL shows 404.
- The bell count updates after an analyst resolves an alert.
- All pages work at 360 px and in both themes.

Commit: `feat(customer): add dashboard, accounts, transfer, beneficiaries and history`

Phase 12 — Analyst and Admin areas

```
PHASE 12. Build the analyst pages under /analyst and the admin pages under
/admin from PRD section 9.
- Analyst: dashboard KPIs (open alerts, average risk score, resolved today,
false-positive rate); alert queue with status and severity filters, severity
badges, sorted by severity then age; alert detail with transaction
information, customer summary, a risk gauge, the rule-hit breakdown, the
customer's recent history, a required notes field, and Release and Confirm
block buttons with confirmation modals.
- Admin: dashboard KPIs and charts (transactions per day, alerts by rule);
users table (search, filters, create analyst or admin, change role, disable
or enable; hide the actions on the admin's own row); accounts (freeze and
unfreeze with confirmation); fraud rules (toggle, edit threshold, window and
points with validation 0 to 100); system settings; audit log viewer with
filters; all transactions.
- The admin sees the alert queue as read-only (no resolve buttons).
- Reuse the shared table, pagination, filter bar, badge and modal components.
```

Checks

- As the analyst, resolve one alert with Release and another with Confirm block; the customer is notified each time.
- As the admin, lower a rule's points and see the next transfer score change; the change appears in the audit log.
- Analyst cannot open `/admin/...`; admin has no resolve buttons.
- Admin cannot disable or demote themselves in the UI or through the API.

Commit: `feat(staff): add fraud analyst and admin consoles`

Phase 13 — Frontend tests and end-to-end smoke test

```
PHASE 13. Add frontend tests.
- Vitest and React Testing Library: Login form (validation and server error),
Register form, Transfer form (validation and idempotency key reuse),
ProtectedRoute and RoleRoute, ThemeContext toggle and persistence, axios 401
refresh interceptor (mock the API), formatCurrency and mask utilities.
- One Playwright smoke test: open the home page, log in as a seeded customer,
see the dashboard, log out. Document how to run it.
- Add npm scripts: lint, typecheck, test, test:coverage, e2e.
```

Checks

- `npm run lint`, `npm run typecheck`, `npm test` and `npm run build` all pass.

- The Playwright smoke test passes against the running stack.

Commit: `test(frontend): add component tests and e2e smoke test`

Phase 14 — Docker, CI/CD and monitoring

```
PHASE 14. Containerise and automate (PRD section 15).
- backend/Dockerfile (multi-stage Maven build, then a slim Java 25 runtime,
non-root user) and frontend/Dockerfile (Node build, then nginx serving dist
and proxying /api to the backend). Add .dockerignore files.
- docker-compose.yml with mysql (9.7 LTS, volume, healthcheck), backend
(waits for healthy mysql, uses environment variables), frontend, mailhog. One
command must start everything: docker compose up --build.
- nginx.conf with security headers (X-Content-Type-Options, X-Frame-Options
DENY, Referrer-Policy and a strict Content-Security-Policy that allows the
app's own media and scripts).
- Jenkinsfile: Checkout, Build, Test, SonarQube scan, Docker build, optional
Deploy. .github/workflows/ci.yml: on every pull request run backend tests,
frontend lint, typecheck, tests and build.
- Restrict Actuator in the prod profile (only health public) and keep Swagger
off or restricted in prod.
- scripts/start-dev.sh, start-dev.ps1 and reset-dev-db.sh.
- Complete the root README (see PRD section 13 for the full list of sections)
with real screenshots placeholders and an architecture diagram.
```

Checks

- On a clean machine state: `docker compose down -v` then `docker compose up --build` brings everything up and the home page loads at the frontend port.
- Register, login and a transfer work through nginx (same origin, cookie works).
- `.env` is not in the images or in Git; the containers run as non-root.
- The GitHub Actions workflow passes on a pull request.

Commit: `ci: add docker, compose, jenkins pipeline and github actions`

Phase 15 — Final audit against the Definition of Done and release v1.0.0

```
PHASE 15. Act as a strict reviewer. Check the finished project against PRD
section 18 (Definition of Done), one item at a time. For each item, run the
real command or action, and report PASS or FAIL with evidence. Fix every FAIL
that is a bug or a deviation from the PRD, rerun the check, and list anything
you could not verify. Also: (1) confirm no secrets or large binaries are in
Git history, (2) confirm every PRD endpoint exists and is documented in
Swagger, (3) confirm every page works in both themes at 360 px, (4) update
docs/DECISIONS.md with every deviation and why, (5) list the remaining known
limitations honestly.
```

Checks: all 12 Definition-of-Done items show PASS with evidence.

Commit and tag: `chore: final audit fixes` then merge to `main` and tag `v1.0.0`.

Phase 16 — Complete documentation and code walkthrough

Paste this phase only after Phase 15. It is your own documentation prompt, kept in full (only the quotation marks around file names were changed to backticks). It produces the 14 documents in `docs/` and the updated README.

```
Task: Create Complete Project Documentation and Code Walkthrough

Act as a senior Java Full Stack developer, software architect, code reviewer,
and technical mentor.

I have built this project using AI coding tools, and I want to understand the
entire project thoroughly so I can independently explain, debug, maintain,
and extend it.

Step 1: Inspect the existing project

Before writing anything:
1. Inspect the complete project directory and its folder structure.
2. Identify the frontend, backend, database, build tools, frameworks,
libraries and external integrations actually present.
3. Inspect the main configuration files, dependency files, source code,
routes, controllers, services, repositories, entities, DTOs, database
scripts, tests and deployment files.
4. Identify the application entry point and determine how the application
starts.
5. Trace the important features from the user interface to the backend and
```

```
database.
6. Inspect Git history, timestamps, logs and other available evidence if
possible to establish the actual development sequence.
7. Clearly distinguish verified facts, reasonable inferences and unknown
information. Do not invent historical development steps.

Do not modify existing source code, configuration, database contents or
application behaviour. Do not expose credentials, API keys, tokens or
passwords in documentation.

Step 2: Create these documentation files

Create a `docs/` directory and generate:
1. `PROJECT_OVERVIEW.md`
2. `PROJECT_STRUCTURE.md`
3. `ARCHITECTURE.md`
4. `DEVELOPMENT_WORKFLOW.md`
5. `BACKEND_CODE_WALKTHROUGH.md`
6. `API_DOCUMENTATION.md`
7. `DATABASE_DESIGN.md`
8. `FRONTEND_BACKEND_FLOW.md`
9. `SECURITY_AND_CONFIGURATION.md`
10. `TESTING_AND_DEBUGGING.md`
11. `DEPLOYMENT_GUIDE.md`
12. `LEARNING_AND_INTERVIEW_GUIDE.md`
13. `GLOSSARY.md`
14. `KNOWN_ISSUES_AND_IMPROVEMENTS.md`

Also update the root `README.md` with a project overview, setup instructions,
run commands and links to these documents. Preserve useful existing README
content.

Step 3: Explain the architecture

Document the actual architecture and include diagrams where useful.

Explain:
- The purpose of each major module.
- The responsibilities of each important file.
- How the modules depend on one another.
- How frontend, backend, database and external services communicate.
- The application's entry point and startup sequence.
- Configuration, dependency injection and request handling.

For a Java Spring Boot backend, if present, explain the actual use of
controllers, services, repositories, entities, DTOs, validation, exception
```

```
handling, Spring Security, JPA/Hibernate and transactions. Do not claim that
a component is used if it does not exist in the project.

Step 4: Explain the complete workflow

For each major feature, provide a numbered, end-to-end execution flow.

Example:
User action → frontend component → API client → HTTP request → controller →
validation → service → repository → database → response DTO → HTTP response →
frontend state update.

Replace this example with the actual files, class names, methods and routes
found in the project.

Include at least:
- Application startup.
- One successful read operation.
- One successful create/update operation.
- Login and authorization, if implemented.
- Error handling.
- Database interaction.
- External API interaction, if present.

Show the sequence in which methods are called and identify which layer
performs each operation.

Step 5: Explain each important file and method

For every important source file, document:
- Exact relative file path.
- Its purpose.
- Why it exists.
- Its important classes, methods and variables.
- Its dependencies and callers.
- Which files call it and which files it calls.
- Inputs, outputs and side effects.
- Relevant framework concepts.
- A concrete example of its execution.
- Common bugs and debugging techniques.

Prioritize the backend and explain the code in small, understandable
sections. Avoid copying entire source files into the documentation.

Step 6: API and database documentation
```

```
For every API, provide:
- HTTP method and route.
- Purpose.
- Authentication and role requirements.
- Path/query parameters.
- Request JSON.
- Successful response JSON.
- Common error responses.
- Validation rules.
- Controller and service methods involved.
- Database tables affected.

For the database, provide:
- Tables, columns and data types.
- Primary and foreign keys.
- Relationships and constraints.
- Entity-to-table mappings.
- Schema or migration locations.
- Safe local development setup and sample data instructions.

Use only schema details verified from the actual project.

Step 7: Explain the development sequence

Describe how the project appears to have been built.

Separate:
A. Verified historical development sequence based on available evidence.
B. Reconstructed sequence based on code dependencies.
C. Recommended sequence for rebuilding the project from scratch.

For each phase, explain its objective, files created or changed,
prerequisites, commands, verification steps and dependencies on earlier
phases.

Do not fabricate Git history or pretend that the exact order is known when
evidence is missing.

Step 8: Setup, execution, testing and deployment

Provide commands appropriate to the project's actual stack and operating
system.

Explain:
- Prerequisites and versions.
- Dependency installation.
```

```
- Environment variable setup using safe placeholders.
- Database creation and configuration.
- Frontend and backend startup.
- Build commands.
- Test execution.
- API testing.
- Common errors and their solutions.
- Deployment and production configuration.

Never print actual secrets.

Step 9: Learning and interview preparation

Create a learning guide tied to the real project.

Include:
- Concepts I need to learn first.
- A dependency-ordered learning path.
- Java, Spring Boot, REST API, SQL and database concepts actually used.
- Important code snippets from the project, with explanations.
- At least 30 project-specific interview questions with detailed answers.
- Debugging exercises and suggested modifications.
- Features I should implement myself to prove that I understand the project.

Step 10: Final quality checks

Before finishing:
1. Verify that documented paths, class names, methods and endpoints actually
exist.
2. Check that run commands match the actual build configuration.
3. Mark missing, incomplete or unimplemented features honestly.
4. Separate current functionality from suggested future improvements.
5. Ensure diagrams match the implementation.
6. Cross-link all documentation files.
7. Provide a summary of which files were created and any questions that could
not be answered from the available evidence.

Do not change application code or perform destructive actions. If
documentation becomes too large, generate the files in manageable batches
while preserving detail.

The goal is that I can understand and explain the project independently, not
merely read a high-level summary.
```

Checks

- All 14 files exist in `docs/`, are cross-linked, and the README links to them.

- Pick 10 random file paths, class names and endpoints from the docs and confirm each exists in the code.
- Run the documented setup commands on a clean clone; they work.
- The docs separate what is verified, inferred and unknown, and mention the Git history only as far as it really exists.
- No secrets appear anywhere in the docs.

Commit: `docs: add complete project documentation and code walkthrough`

Recovery prompts (use when something goes wrong)

The build or a test fails

```
The build/test failed. Do not change the tests to make them pass unless the test itself is wrong. Show me the exact error, explain the root cause in two sentences, fix the real cause, rerun the same command, and show me the passing output. Do not touch files outside the failing area.
```

The agent drifted from the PRD

```
Stop. Compare what you built in this phase with docs/PRD.md. List every difference (names, paths, tables, endpoints, behaviour, security rules). For each difference, either fix it to match the PRD or explain why the PRD is wrong and add the decision to docs/DECISIONS.md. Do not add features that are not in the PRD.
```

A dependency or API looks invented

```
Verify every dependency, annotation and class you used in this phase against official documentation for the exact versions in the project. List anything you could not verify. Remove or replace anything that does not exist. Show the source you used for each fix.
```

I do not understand a file

```
Explain <file path> to me like a mentor preparing me for a viva. Cover: why it exists, what each method does step by step, who calls it, what it calls, one concrete example with real values, the framework concepts involved, three likely viva questions with answers, and one bug that could happen here and how to debug it. Do not change any code.
```

Security self-check (run after Phases 3, 6, 7 and 12)

```
Review the code from this phase as a security auditor. Check: authentication bypass, missing role checks, IDOR and ownership checks, mass assignment, injection, sensitive data in logs and responses, token handling, CORS, rate limiting, and error messages that leak information. For each finding give the file, the risk, a fix, and a test that proves the fix. Apply the fixes and rerun all tests.
```

Final submission checklist

- [ ] Project title is exactly **TrustPay Guardian: Fraud-Aware Digital Banking Platform** in the README, UI, Swagger and report.
- [ ] All 16 phases are committed, merged and tagged (`v0.1.0` , `v1.0.0`).
- [ ] Home page videos are your own or properly licensed, and the licences are listed.
- [ ] You can demo the full script in section 16 of the PRD without help.
- [ ] You can explain: JWT flow, refresh rotation, RBAC and ownership checks, idempotency, row locking, the fraud rules and scoring, and the Strategy pattern used for rules.
- [ ] The Technical Report (RBAC and JWT security) uses the real security design and test results from this project, written in your own words and checked for similarity.